

hermetic_wg_roundtrip.sh — companion guide (§11.4.18)

Revision: 2 **Last modified:** 2026-07-02T00:00:00Z **Status:** H0-**full** for the hermetic WireGuard test harness ([design](#)).
Authority: inherits constitution/Constitution.md per §11.4.35.

Overview

tests/vpn_lan/hermetic_wg_roundtrip.sh upgrades the H0 veth PoC ([hermetic_netns_poc.sh](#)) to a **real encrypted kernel-WireGuard tunnel** between two unprivileged network namespaces, and proves a real HTTP payload round-trips **over the tunnel** (not the underlay):

```
netns A (in the userns)                netns B (peer / holder)
veth0 10.9.0.1 — underlay (UDP) — veth1 10.9.0.2    <-
carries encrypted WG
wg0 10.10.0.1 == WireGuard tunnel == wg0 10.10.0.2  <- the
overlay
python3 http.server
bound to 10.10.0.2
```

The peer HTTP server binds to 10.10.0.2 — a WG-**only** overlay address routable exclusively through the tunnel (allowed-ips 10.10.0.2/32). A successful fetch to it therefore requires a completed WireGuard handshake.

Everything is **fully unprivileged**: unshare -Ur maps the caller to root inside a throwaway user namespace, where the host wireguard kernel module + /usr/sbin/wg create and configure wg0. **No build, no wireguard-go, no podman, no Mullvad, no package install.** (The design originally planned a userspace wireguard-go build; the kernel-WG-in-userns path proved feasible and is used instead — simpler and faster, §11.4.82.)

Prerequisites

- bash, util-linux unshare + nsenter, iproute2 ip (with the wireguard link type), wireguard-tools wg, python3, sha256sum, timeout (self-bounds the peer server inside the netns so an orphan terminates on its own, §12).

- The host wireguard **kernel module loaded** (/sys/module/wireguard).
- Unprivileged user namespaces permitted + process headroom (§12).

Any missing prerequisite → honest SKIP: (exit 0), never a fake PASS (§11.4.3).

Usage

```
tests/vpn_lan/hermetic_wg_roundtrip.sh          # PASS /
          SKIP / FAIL
WG_MUT=badkey tests/vpn_lan/hermetic_wg_roundtrip.sh # §1.1
          golden-bad
```

Evidence: qa-results/vpn_lan/hermetic_wg/<UTC-ts>_<pass|mut>_<pid>/roundtrip.evidence.

Anti-bluff design (why this proves the tunnel, not the underlay)

Two independent oracles, both required for a normal PASS:

1. **sha256 payload round-trip** — the served nonce is fetched over 10.10.0.2 and verified byte-for-byte.
2. **wg show handshake + transfer** — latest-handshake != 0 **and** both rx > 0 and tx > 0, proving a real WireGuard session moved bytes.

The **golden-bad** (WG_MUT=badkey) gives netns A a *wrong* peer public key. The only acceptable outcome is the round-trip **failing at an incomplete handshake**. The load-bearing signal is **rx = 0**: with a wrong key nothing decrypts, so no bytes come back — if traffic were leaking over the veth underlay in the clear, a wrong WG key would not matter and rx would be non-zero. This is the §11.4.107(10) / §11.4.68 proof that the WireGuard crypto genuinely gates the traffic.

Captured evidence (verified 2026-07-02)

- **Normal PASS:** wg show wg0: latest-handshake=1782947082 rx=452 tx=752 (peer count=1); sha256 source == fetched (55dca799...). **3/3 deterministic** (§11.4.50).

- **Golden-bad:** latest-handshake=0 rx=0 tx=888 → WG_FAIL: no connect over 10.10.0.2:8080 (WG handshake incomplete) — rx=0 confirms the crypto is load-bearing.

Security / keys (§11.4.10)

WireGuard private keys are generated into mode-0600 mktemp files **inside the throwaway namespace**, used by path (wg set private-key <file>), and removed on exit. They are **never** printed, logged, or committed. Only public keys and transfer counters appear in the evidence.

Edge cases

- Kernel module absent / wg absent / userns disabled / low headroom → honest SKIP.
- Setup race prevented by the holder marker (see the veth PoC guide).
- WG_MUT=badkey that somehow still round-trips → reported as "may be leaking over the underlay" FAIL (never a silent pass).

Related

- [hermetic_netns_poc.md](#) — the veth substrate this builds on.
- [hermetic_wg_test_harness.md](#) — the design (H1/H2/H3 next).
- tests/lib/sword_bridge.sh — the HELIX_BRIDGE_MODE=hermetic integration target (H2).

Last verified

2026-07-02 (host: ALT Linux kernel 6.12, wireguard module loaded, wireguard-tools 1.0.20210914, util-linux 2.39.2, python3).